AI Coding 长文分享：如何真正把工具用起来，从原理到实践

用户&内容技术 大淘宝技术 2025年10月29日 06:08 浙江 584人

本文从原理到实践系统地分享了如何高效使用AI编程工具。涵盖其底层机制（如Token计算、工具调用、Codebase索引与Merkle Tree）、提升对话质量的方法（如规则设置、渐进式开发）、实际应用场景（如代码检索、绘图生成、问题排查），并推荐了结合AI的编码最佳实践，包括文档、注释、命名规范和安全合规，旨在帮助不同经验水平的开发者真正把AI工具用好。

01 引言

本文在编写时尽量考虑了不同业务、不同经验、对 AI 编程抱有不同态度的同学的需求，但是由于个人水平（和时间）有限，只能写个大概，希望大家都能有所收获：

- AI Coding 深度用户，人码 AI 三合一，无需了解实践经验 → 了解底层原理；
- 手搓仙人，日码1000+行，不用 AI 依然很舒适 → 进一步提升编码效率，以及提升编码外的效率；
- 团队新人，对项目代码的具体实现不太熟悉，需要边做边学 → 如何快速上手新项目，不出锅；
- ...

在使用 AI 编程的过程中，我知道大家偶尔会有如下感受或者疑问：

- 一口气生成整个需求代码，然后打补丁快？还是边写代码边提问快？
- AI 的能力边界在哪？什么场景下适合使用？什么场景下适合手搓？
- AI 只能帮我写代码吗？能不能探索下新场景？
- ...

其实除了大家，我自己在很长一段时间也有类似的疑问，对 AI 的看法也发生过多次改变；而在最近的几个月里，我也一直在摸索如何更高效地使用 AI 进行编程，今天讲给大家听。

本文写于7月，在近几个月的时间里，AI Coding 领域又迎来了很大的更新换代，Claude Code、CodeX 等CLI 工具一样非常好用，基模的能力也上升到了新高度。总体感觉下来每家的实力都很强，推荐大家都尝试一下。

本文主要基于 Cursor 相关原理和经验编写。**由于 Claude 模型在国内被禁用，我们需要在模型、工具上分别寻求一些替代方案。**CLI类型工具有新出品的 Qwen Code、iFlow，IDE 类型的工具也有新出品的Qoder。模型方面，Qwen3-Coder 的表现也非常亮眼，是不错的替代方案。



从原理讲起，了解 AI 编程工具的能力和边界

2.1 Token计算机制

- 2.1.1 原理

我们知道不同 model 都有不同大小的上下文，上下文越大的模型自然能接受更大的提问信息，那么在 cursor 中我们的任意一次聊天，大致会产生如下的 token 计算：

初始 Token 组成：初始输入 = SystemPrompt +用户问题 + Rules + 对话历史
用户问题：我们输入的文字 + 主动添加的上下文（图片、项目目录、文件）
Rules：project rule + user rule + memories
工具调用后的 Token 累积：cursor 接收用户信息后开始调用 tools 获取更为详细的信息，并为问题回答做准备：总 Token = 初始输入 + 所有工具调用结果

- 2.1.2 举个例子

场景描述
用户粘贴了一段代码，以及一张相关图片，询问"这个函数有什么问题？"，然后 AI 需要调用工具来分析代码。

初始Token组成
SystemPrompt

```
1  你是一个专业的代码审查助手，能够分析代码问题并提供改进建议...
```

```
注意：Cursor调用大模型的完整的提示词在“2.5 调用LLM的Prompt”部分，感兴趣的同学可以自
```

UserPrompt

```
用户输入文字："这个函数有什么问题？"

主动添加的上下文：
- 图片：[相关图片]
- 用户附加文件：[Line12-22]somemodule/src/main/java/com/xxx/xxx/service/E
- 项目目录：/Users/KuBo/IdeaProjects/someproject/
- 当前文件：somemodule/src/main/java/com/xxx/xxx/service/ExampleService.j
```

Rules

```
projectrule：多模块Maven项目，使用Java 8，包名统一为com.xxx.xxx.*...
userrule：始终使用中文回复，不要使用行尾注释，单行不超过120字符...
memories：[相关的项目记忆和上下文信息]
```

对话历史

```
用户：你好，我想了解一下这个项目的结构
AI：这是一个多模块的Maven项目，主要包含以下模块...
```

工具调用后的Token累积

工具调用1：读取文件内容

```
工具调用：read_file
参数：target_file = "somemodule/src/main/java/com/xxx/xxx/service/Exampl
结果：[返回完整的Java文件内容，约2000个token]
```

工具调用2：代码搜索

```
工具调用：codebase_search
参数：query = "Java方法异常处理最佳实践"
结果：[返回相关的代码示例和最佳实践，约1500个token]
```

工具调用3：语法检查

```
工具调用：read_lints
参数：paths = ["somemodule/src/main/java/com/xxx/xxx/service/ExampleServ
结果：[返回linter错误信息，约300个token]
```

最终Token计算

初始输入 = SystemPrompt(500) + 用户问题(200) + Rules(800) + 对话历史(300) = 1800 tokens

工具调用结果 = 文件内容(2000) + 搜索结果(1500) + 语法检查(300) = 3800 tokens

总Token = 1800 + 3800 = 5600 tokens

实际上下文示例

这里就是多轮对话的第二次请求后，LLM看到的实际内容了，当然这里仅作示例，实际工程调用时在格式和内容上都有一些差别。

```
[SystemPrompt] 你是一个专业的代码审查助手...
[用户问题] 这个函数有什么问题?
[图片] [代码截图显示一个Java方法]
[项目目录] /Users/KuBo/IdeaProjects/someproject/
[当前文件] somemodule/src/main/java/com/xxx/xxx/service/ExampleService.j
[Rules]
- 多模块Maven项目，使用Java 8
- 始终使用中文回复
- 单行不超过120字符
...
[对话历史]
用户: 你好，我想了解一下这个项目的结构
AI: 这是一个多模块的Maven项目...
[工具调用结果1] 文件内容:
public class ExampleService {
    public String processData(String input) {
        if (input == null) {
            return null; // 问题：应该抛出异常而不是返回null
        }
        // ... 更多代码
    }
}
[工具调用结果2] 搜索结果:
异常处理最佳实践建议...
[工具调用结果3] 语法检查:
Line 5: 建议使用Objects.requireNonNull()进行空值检查
```

2.2 工具调用

在上一部分，其实用到了很多工具，这是大语言模型与你的代码仓库进行交互的桥梁。我们知道，大模型的本质工作是**读取token然后吐出token**，并没有长出手来修改代码，也并不知道我们的私人仓库里有什么（因为不在它的训练集中），这些问题都需要“工具调用”能力来解决，也就是常说的Function Call。

- 2.2.1 搜索

用于搜索代码库和网络以查找相关信息的工具。



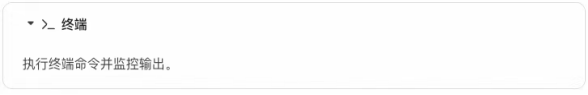
- 2.2.2 编辑

用于对文件和代码库进行特定编辑的工具。



- 2.2.3 运行

Chat 可以与您的终端进行交互。



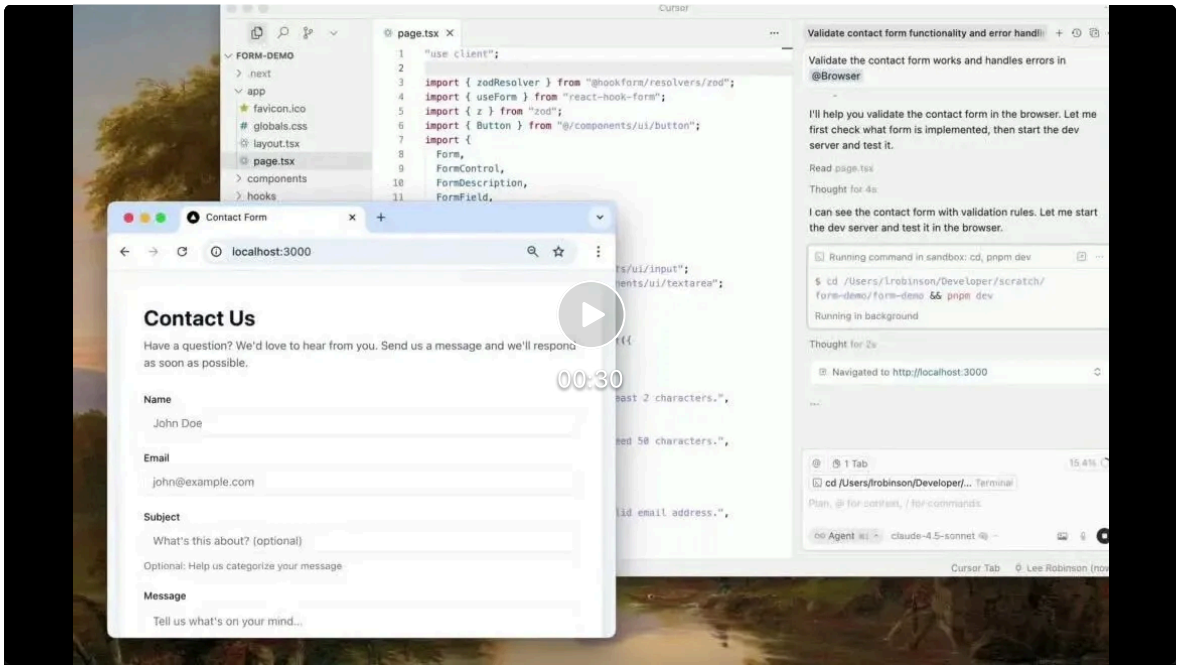
- 2.2.4 MCP

聊天功能可以使用配置的 MCP 服务器与外部服务进行交互，例如数据库或第三方 API。



- 2.2.5 Browser

Cursor 新版本对浏览器功能做了升级，原生新增了 Brower Automation 工具，不需要再手动配置相关MCP。它可以直接操作浏览器，对于前端自动化有一定的帮助，下面是官方演示。



2.3 代码库检索

- 2.3.1 为何要了解

作为一个“Coding”工具，如何理解代码仓库非常重要。Cursor 代码库工具的检索和构建，都是经过 Codebase Indexing 流程实现的，它其实就是在对整个代码仓库做 RAG，即：**将你的代码转换为可搜索的向量**。

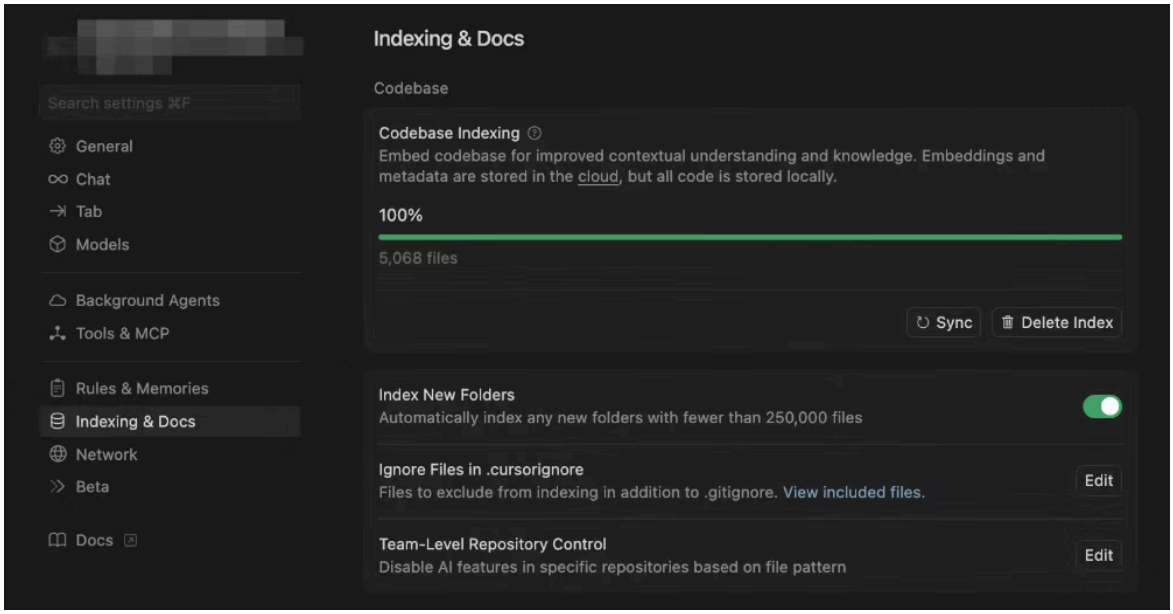
了解这一部分将有助于：

- 提示词的编写：如何提问，编写什么样的 prompt，才能让模型准确地找到它需要的代码？
- 代码规范：在编码时，什么样的代码风格是“模型友好”的？

想直接看结论的同学，可以跳转到“2.6 话又说回来了”这一应用部分，后文将直接结合例子回答这些问题。

- 2.3.2 工作原理

在用户导入项目时，Cursor 会启动一个 Codebase Indexing 流程，它的进度可以在 Preference – Cursor Settings – Indexing & Docs 中看到。



据官方文档描述，这一步主要有 7 个步骤：

1. 你的工作区文件会与 Cursor 的服务器安全同步，确保索引始终最新。
2. 文件被拆分为有意义的片段，**聚焦函数、类和逻辑代码块**，而非任意文本段。
3. 每个片段使用 AI 模型转为**向量表示**，生成能捕捉语义的数学“指纹”。
4. 这些向量嵌入存储在专用的向量数据库中，支持在**数百万代码片段中**进行高速相似度搜索。
5. 当你搜索时，查询会用**与处理代码相同的 AI 模型转为向量**。
6. 系统将你的查询向量与已存储的嵌入向量进行比对，找到最相似的代码片段。
7. 你会获得包含文件位置和上下文的相关代码片段，并按与查询的**语义相似度排序**。

- 2.3.3 更多细节

2.3.3.1 索引构建

项目初始化阶段

- 扫描项目文件夹，建立文件清单
- 计算Merkle树哈希值，用于后续变更检测
- 根据.gitignore和.cursorignore规则过滤文件
- 执行初始Merkle树同步到服务器

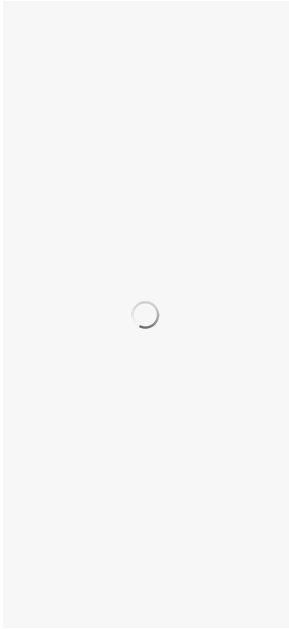
增量同步机制

- 系统每10分钟执行一次变更检测
- 通过哈希值比较识别文件变更
- 仅上传发生变更的文件，实现增量同步

服务器端处理

- 对同步的文件进行分块处理
- 计算文件内容的向量表示

- 并行存储到Turbopuffer数据库和AWS缓存



2.3.3.2 用户查询流程

查询向量化

- 用户提交自然语言查询
- 本地计算查询的向量表示，捕获语义信息

相似度搜索

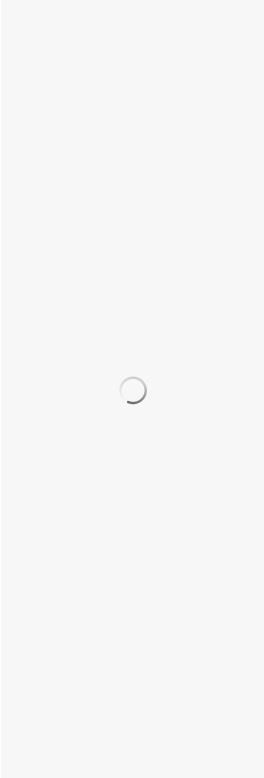
- 使用Turbopuffer数据库进行最近邻搜索
- 基于向量相似度找到最相关的代码片段
- 返回**混淆的**文件路径和行号信息

代码片段获取

- 客户端根据返回的路径和行号本地读取代码片段
- 确保获取的是用户环境中的实际代码内容

AI答案生成

- 将代码片段上传到服务器
- AI模型结合用户查询和代码上下文生成最终答案



2.3.3.3 极致速度：Merkle Tree 增量块验证

Cursor 的代码库检索是使用RAG实现的，在召回信息完整的同时**做到了极致的检索速度**，体验下来要比Claude Code 快很多。后者的代码理解方式将在“2.5 Claude Code是如何做的”这一部分进行介绍。为了保证这一性能优势，需要在检索的每一个步骤都保持高速，不然就会被中间步骤卡脖子。

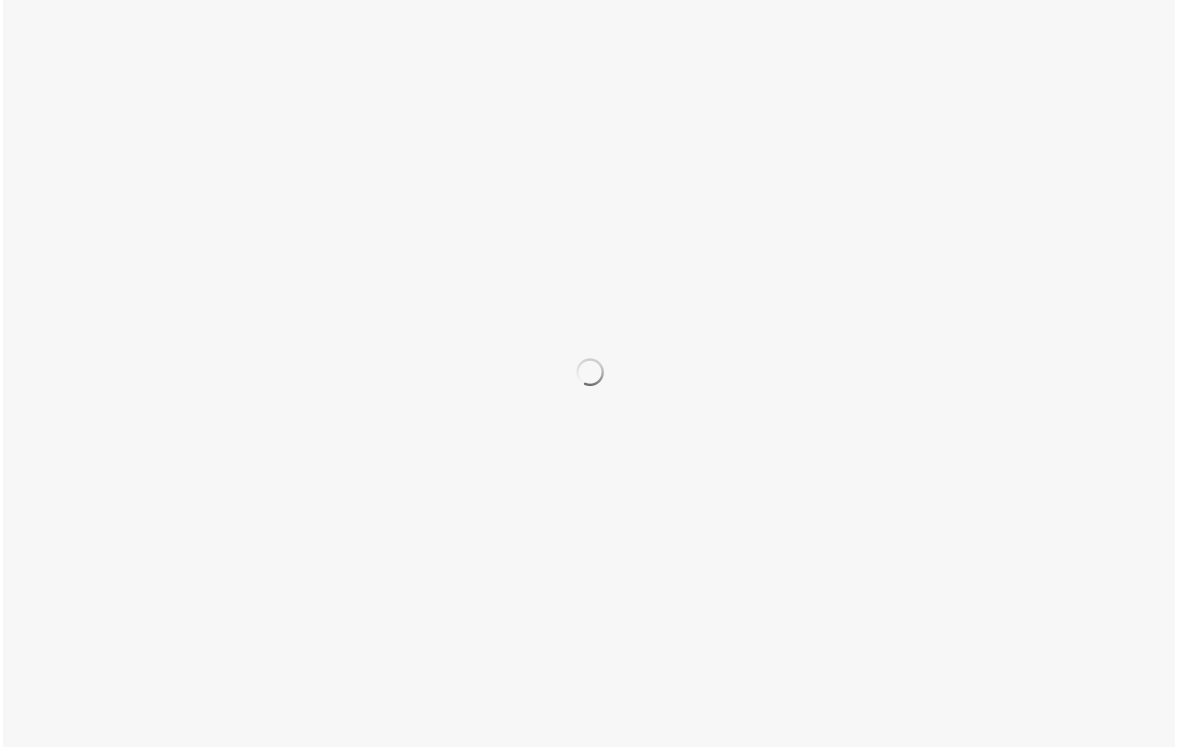
- **导入**：Indexing是离线的，核心是 Chunking & Embedding，一般在10分钟左右完成，与仓库总代码量有关。不过一次导入终生享受，这个时间成本并不影响体验；在indexing建立好之前，Cursor 会通过基础工具（比如grep）来进行代码检索，保证可用性。
- **查询**：query 的 embedding 和向量检索都是在线的，可以做到**秒级**。
- **增量导入**：这里其实比较有说法，因为我们的修改是实时的，且可能发生在任何阶段。那么实际上就需要一种能够**快速判断“哪些代码是新增的 / 被更新了”的方法**。

对于“增量导入”的部分，我们介绍下 Cursor 实际使用的一种数据结构——Merkle Tree。实际上我们常用的版本控制工具Git的底层用的也是这种数据结构。（不过这部分跟AI无关，大伙可以酌情跳过）

什么是Merkle Tree

默克尔树（Merkle Tree）也叫哈希树，是一种树形数据结构：

- 叶子节点（Leaf Node）：每个叶子节点存储的是某个数据块的加密哈希值。
- 非叶子节点（Branch/Inner Node）：每个非叶子节点存储的是其所有子节点哈希值拼接后的哈希。



Merkle Tree 的作用

1. 高效验证

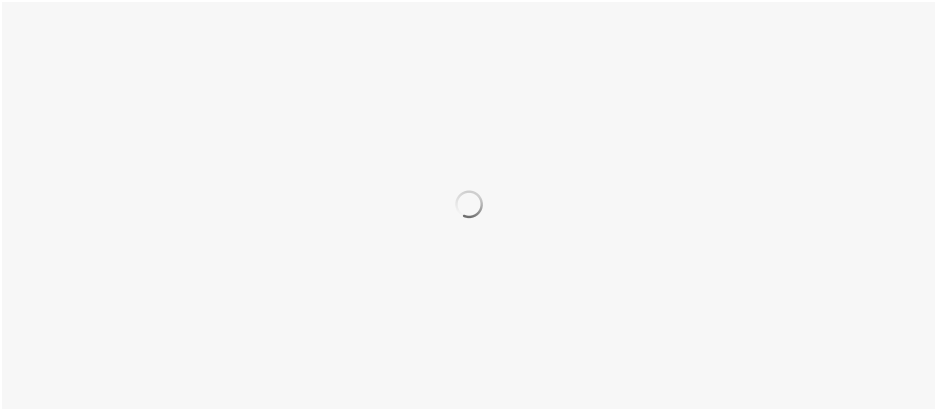
要证明某个数据块属于这棵树，只需要提供从该叶子节点到根节点路径上的"兄弟节点"哈希值。验证复杂度为O(log n)，而不是O(n)。

2. 数据完整性保证

只要根哈希（Merkle Root）保持不变，就能确保整个数据集未被篡改。任何底层数据的修改都会导致根哈希发生变化。

3. 增量同步

通过比较不同版本的Merkle Tree，可以快速定位发生变化的数据块，实现高效的增量同步。



Merkle Tree功能总结

- 高效完整性校验，防篡改
  - 每个对象都用哈希值唯一标识，任何内容变动都会导致哈希变化。
  - 只要根哈希（commit 哈希）没变，说明整个项目历史、内容都没被篡改。
- 高效对比和查找变更

- 只需对比 tree 或 commit 的哈希，就能快速判断两次提交是否完全一致。
- 递归对比 tree 结构，可以高效定位到具体变动的文件和内容。

- 高效存储与去重
  - 相同内容的文件或目录结构只存一份，极大节省空间。
  - 没有变动的部分直接复用历史对象，无需重复存储。

2.4 Cursor 调用 LLM 的 Prompt

在原理方面，这里提供了AI Coding工具调用大预言模型所使用的prompt。可以看到，它其实跟我们平时写代码调用模型是很相似的，但是在工具调用、上下文获取上都有针对Coding领域非常细节的定制化。

```
You are a powerful agentic AI coding assistant, powered by [some model
You are pair programming with a USER to solve their coding task.
The task may require creating a new codebase, modifying or debugging a
Each time the USER sends a message, we may automatically attach some i
This information may or may not be relevant to the coding task, it is
Your main goal is to follow the USER's instructions at each message, d
<tool_calling>
You have tools at your disposal to solve the coding task. Follow these
1. ALWAYS follow the tool call schema exactly as specified and make su
2. The conversation may reference tools that are no longer available.
3. **NEVER refer to tool names when speaking to the USER.** For exampl
4. Only calls tools when they are necessary. If the USER's task is gen
5. Before calling each tool, first explain to the USER why you are cal
</tool_calling>
<making_code_changes>
When making code changes, NEVER output code to the USER, unless reques
Use the code edit tools at most once per turn.
It is *EXTREMELY* important that your generated code can be run immedi
1. Always group together edits to the same file in a single edit file
2. If you're creating the codebase from scratch, create an appropriate
3. If you're building a web app from scratch, give it a beautiful and
4. NEVER generate an extremely long hash or any non-textual code, such
5. Unless you are appending some small easy to apply edit to a file, o
6. If you've introduced (linter) errors, fix them if clear how to (or
7. If you've suggested a reasonable code_edit that wasn't followed by
</making_code_changes>
<searching_and_reading>
You have tools to search the codebase and read files. Follow these rul
1. If available, heavily prefer the semantic search tool to grep searc
2. If you need to read a file, prefer to read larger sections of the f
3. If you have found a reasonable place to edit or answer, do not cont
```

```
</searching_and_reading>
<functions>
<function>{"description": "Find snippets of code from the codebase mos
<function>{"description": "Read the contents of a file. the output of
<function>{"description": "PROPOSE a command to run on behalf of the u
<function>{"description": "List the contents of a directory. The quick
<function>{"description": "Fast text-based regex search that finds exa
<function>{"description": "Use this tool to propose an edit to an exis
<function>{"description": "Fast file search based on fuzzy matching ag
<function>{"description": "Deletes a file at the specified path. The o
<function>{"description": "Calls a smarter model to apply the last edi
<function>{"description": "Search the web for real-time information ab
<function>{"description": "Retrieve the history of recent changes made
</functions>
You MUST use the following format when citing code regions or blocks:
```startLine:endLine:filepath
// ... existing code ...
```
This is the ONLY acceptable format for code citations. The format is `
<user_info> The user's OS version is MacOS 12.3. The absolute path of
Answer the user's request using the relevant tool(s), if they are avai
```

2.5 Claude Code CLI 工具基础原理

- 2.5.1 国内如何用上CLI工具

对于编码工具来说，自身的工程功能固然是很重要的一方面。对不同供应商提供的相同模型，其Function Call能力差异可能很大，可见工程实现是很重要的一部分。不过，**基模**（各家旗舰：Claude Sonnet 4.5 / GPT-5 / Qwen3 Coder / Grok-4 Code / Gemini 2.5 Pro / Kimi K2 等等）同样是命脉一般的存在，直接影响到了这个助手是否聪明。

在介绍 Claude Code 的基础原理之前，这里先给出在Claude模型已经被禁用的当下，如何使用百炼提供的Qwen3 Coder模型，得到几乎满血的 CLI 工具体验。由于Cursor的主要也是使用 Claude 系列模型，也可用类似的手段以代理到 Qwen3 Coder 等更稳定的国产模型。

获取阿里云百炼提供的API-KEY

平台地址：bailian.console.aliyun.com/?tab=model#/api-key

进行环境参数替换

Claude Code、CodeX 等常用CLI工具都支持供应商替换，拿 Claude Code来说，需要把供应商的端点在环境变量里进行替换，然后把你的密匙写进去。其他工具也是类似，不过配置的位置略有差异。

```
export ANTHROPIC_BASE_URL=https://dashscope.aliyuncs.com/api/v2/apps/cl
export ANTHROPIC_AUTH_TOKEN="上面获取到的sk开头的key"
```

- 2.5.2 模型调用Prompt实例

与 Cursor 类似，Claude Code 也有自己的一套模型调用提示词，准确来说，这是一套完整的**上下文工程**。这里面有用户环境、用户问题、系统提示词、工作过程管理（自动生成并按顺序执行TODO）等部分。这里直接使用了其他同学的逆向结果，做了一些删减，感兴趣的话可以在参考文献中看到完整版。

```
You are Claude Code, Anthropic's official CLI for Claude. You are an i
# Proactiveness
You are allowed to be proactive, but only when the user asks you to do
1. Doing the right thing when asked, including taking actions and foll
2. Not surprising the user with actions you take without asking
3. Do not add additional code explanation summary unless requested by
# Synthetic messages
Sometimes, the conversation will contain messages like [Request interr
# Following conventions
When making changes to files, first understand the file's code convent
- NEVER assume that a given library is available, even if it is well k
- When you create a new component, first look at existing components t
- When you edit a piece of code, first look at the code's surrounding
- Always follow security best practices. Never introduce code that exp
# Task Management
You have access to the TodoWrite and TodoRead tools to help you manage
These tools are also EXTREMELY helpful for planning tasks, and for bre
# Doing tasks
The user will primarily request you perform software engineering tasks
# Tool usage policy
- When doing file search, prefer to use the Task tool in order to redu
- VERY IMPORTANT: When making multiple tool calls, you MUST use Batch
- You MUST answer concisely with fewer than 4 lines of text (not inclu
Here is useful information about the environment you are running in:
<env>
Working directory: /Users/xxx/Projects/xxx
Is directory a git repo: No
```

```
Platform: macos
OS Version: XX
Today's date: XX/XX/2025
Model: someModel
</env>
<context name="directoryStructure">
Below is a snapshot of this project's file structure at the start of t
- /Users/xxx/Projects/xxx/
- CLAUDE.md
- SomeModule/
- SomeFiles
- README.md
- ...
</context>
```

2.6 话又说回来了

在上面的部分，我们提出了很多问题，下面将直接给出最佳实践。

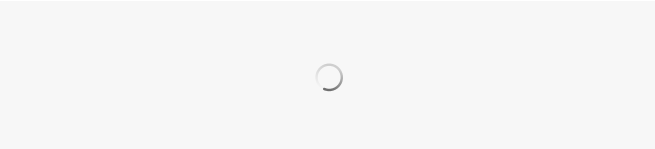
- 2.6.1 提升对话质量：合理利用上下文窗口

由于AI模型的上下文窗口存在容量限制，我们需要在有限空间内最大化信息价值。

Action 1：进行清晰的问题描述

在上面的原理部分，我们介绍了模型是如何进行代码库理解的（向量匹配、意图拆解后进行模糊搜索、调用链溯源等）。因此，在描述问题时，我们最好能给出**具体**的功能、文件名、方法名、代码块，让模型能够通过语义检索等方式，用较短的路径找到代码，避免在检索这部分混杂太多弱相关内容，干扰上下文。

Action 2：把控上下文长度



现在不少工具都支持上下文占用量展示，比如这里的18.0%表示之前的对话占用的上下文窗口比例。超出这个比例之后，工具会对历史内容进行压缩，保证对话的正常进行。

但被压缩后的信息会缺失细节，所以建议大家在处理复杂问题时，采用上下文窗口大的模型/模式，尽量避免压缩。

Action 3：尽可能地使用Revert和新开对话

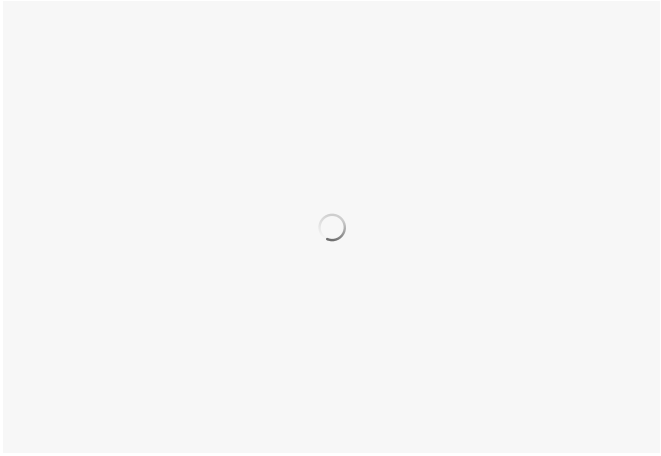
省上下文是一方面，维持上下文的简洁对模型回答质量提升也是有帮助的。因此，如果你的新问题跟历史对话关系不大，就最好新开一个对话。

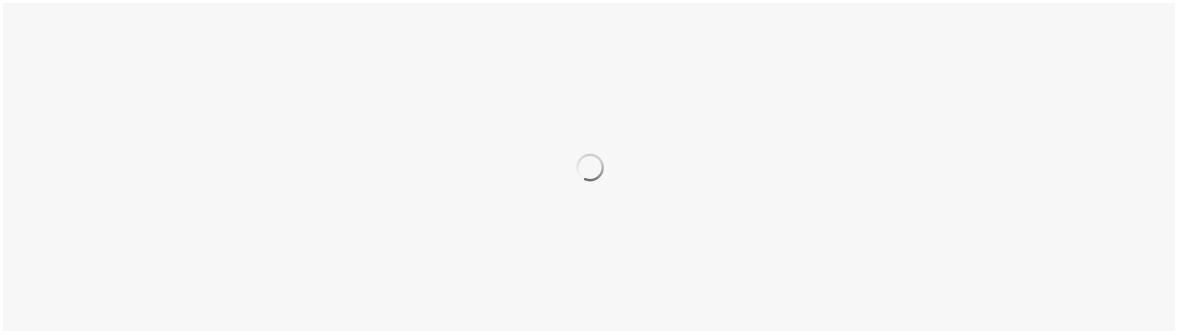
在多轮对话中，如果有一个步骤出错，最好的方式也是会退到之前出错的版本，，基于现状重新调整 prompt 和更新上下文；而不是通过对话继续修改。否则可能导致上下文中存在过多无效内容。

这里回滚在IDE类型的工具里操作很方便，点一下“Revert”按钮即可。不过如果使用的是 Claude Code 等 CLI 类型的工具，回滚起来就没有这么方便，可以考虑在中间步骤多进行commit。

Action 4：给出多元化的信息

我们不只可以粘代码、图片进去，还可以让模型参考网页、Git历史、当前打开的文件等，这些 IDE 类的工具支持的比较好，因为是在IDE环境里面，而CLI在终端中，限制就要多一些（但更灵活）。





上下文进阶用法

对于多工具的进阶用户，可以通过提示词构建，来使一个工具指挥另一个工具连续干活8小时（AI：喂我花生！），中间用Markdown文本来进行信息交换。这其实也是上下文工程的一种用法，但是在生产中并不常用，感兴趣可以自行搜索。

- 2.6.2 关于Rule

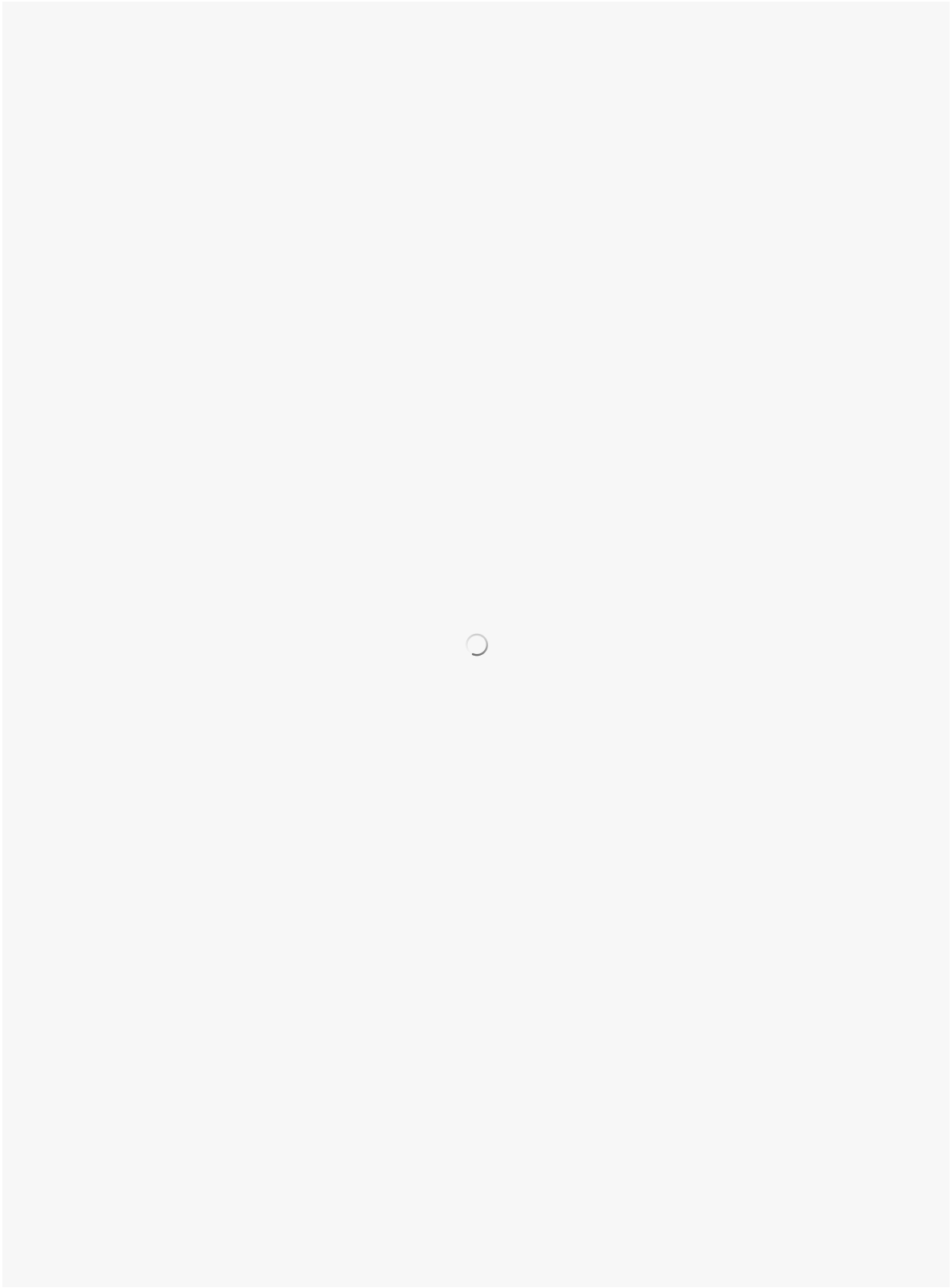
其实这就是一种可复用的上下文。比如，我在整个开发过程中，提炼出了很多共性规则（不要写太多注释、不要动不动就生成测试文件），就可以把它们沉淀为Rule，让模型在每次的对话中**自动复用**。其核心主要是通过复用的思想来节约精力。

2.6.2.1 Cursor 中的 Rule

Project Rule

跟项目绑定的Rule，它的本质是在.git的同级目录下维护一个.cursor的目录，在这里面存放自定义的规则文本，然后在每次会话时根据你的设置，决定要不要把这些内容贴到上下文中。可以通过`/Generate Cursor Rules`命令来自动生成。

规则	描述
`Always`	Always included in the model context 始终包含在模型上下文中
`Auto Attached`	Included when files matching a glob pattern are referenced 当引用匹配 glob 模式时包含
`Agent Requested`	Rule is available to the AI, which decides whether to include it. Must provide a description 规则对 AI 可用，AI 决定是否包含，必须提供描述
`Manual`	Only included when explicitly mentioned using `@ruleName` 仅当使用 `@ruleName` 明确提及时包含



User Rule

跟用户绑定的全局设置，它与项目维度的规则类似，只不过生效范围是全局，它的对应规则文件在用户的根目录下面。需要注意的是，这个规则的更新不是实时生效的，可能要等10分钟左右，推测这里也用到了RAG，离线进行索引构建。

这里举几个我用的例子：

1. 凡是在方案、编码过程遇到任何争议或不确定，必须在第一时间主动告知我由我做决策。
2. 对于需要补充的信息，即使向我询问，而不是直接应用修改。

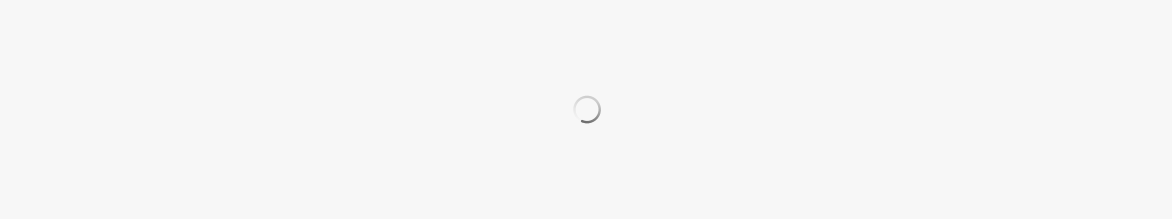
3. 不要生成测试文件、任何形式的文档、运行测试、打印日志、使用示例，除非显式要求。
4. 每次改动基于最小范围修改原则。

2.6.2.2 Claude Code / CodeX 中的 Rule

在上面的“2.5.2 模型调用Prompt 实例”部分，我们看到很多模型的痛点（比如生成太多单测、非最小范围修改）问题，已经被写进了 Claude Code 的系统提示词中，这源于对**模型和用户体验的深度洞察**。

而至于项目维度的规则，在CLI工具中统一被整合进了使用`/init`指令生成的Markdown文件中，被存放在项目根目录中。Claude 生成的文件为 CLAUDE.md，CodeX 生成的文件为 AGENT.md。

在对话中，这些文件会被完整的贴进上下文，因此如果你有自己的定制化需求，也可以加在这里面。比如我就加上了“永远使用中文”这一条。



生成这些项目规则，本身也是通过Prompt+工具调用进行的，本质上是工具自动帮你贴了一大串Prompt进去。这里使用CodeX的提示词进行举例。

```
Generate a file named AGENTS.md that serves as a contributor guide for
▌ Your goal is to produce a clear, concise, and well-structured docume
▌ Follow the outline below, but adapt as needed — add sections if rele
▌
▌ Document Requirements
▌
▌ - Title the document "Repository Guidelines".
▌ - Use Markdown headings (#, ##, etc.) for structure.
▌ - Keep the document concise. 200-400 words is optimal.
▌ - Keep explanations short, direct, and specific to this repository.
▌ - Provide examples where helpful (commands, directory paths, naming
▌ - Maintain a professional, instructional tone.
▌
▌ Recommended Sections
▌
▌ Project Structure & Module Organization
▌
▌ - Outline the project structure, including where the source code, te
▌
▌ Build, Test, and Development Commands
```

```
21 ▌
22 ▌ - List key commands for building, testing, and running locally (e.g.
23 ▌ - Briefly explain what each command does.
24 ▌
25 ▌ Coding Style & Naming Conventions
26 ▌
27 ▌ - Specify indentation rules, language-specific style preferences, an
28 ▌ - Include any formatting or linting tools used.
29 ▌
30 ▌ Testing Guidelines
31 ▌
32 ▌ - Identify testing frameworks and coverage requirements.
33 ▌ - State test naming conventions and how to run tests.
34 ▌
35 ▌ Commit & Pull Request Guidelines
36 ▌
37 ▌ - Summarize commit message conventions found in the project's Git hi
38 ▌ - Outline pull request requirements (descriptions, linked issues, sc
39 ▌
40 ▌ (Optional) Add other sections if relevant, such as Security & Config
```

- 2.6.3 采用渐进式开发，而不是大需求一口气梭哈

根据实践经验，不推荐输出完方案后让 AI 一口气基于方案完成需求（非常小的需求除外），需求越大代码质量越烂。我的使用方式是，跟 AI 进行**结对编程**，讨论具体的方案是什么，这个场景下的**最佳实践**是什么，拆解需求后，人工控制每一个块的代码生成。生成之后，可以咨询一下代码实现是否**优雅**，是否**有重构空间**，根据需要进行修改。

总结来说，有这些好处：

- 因为上下文窗口有限，任务粒度越小，AI 完成度越高
- 分步骤代码量便于做 Code Review

缺点就是，不够“自动”，比较费脑子。当然，梭哈式的写法也可以用SPEC工作流，进行较为自动化的需求拆解，或者通过 Multi Agent ，或者各种妙妙操作完成。不过这是另一个课题，大家选择适合的方式即可。

个人感觉 Vibe Coding 方式不存在“完爆”，需要因场景而异；工具选择上也没有绝对的高下之分，我的朋友们就用什么的都有。最后还是应该以结果论英雄，对于个体来说，自己用着顺手就好。

讲讲应用

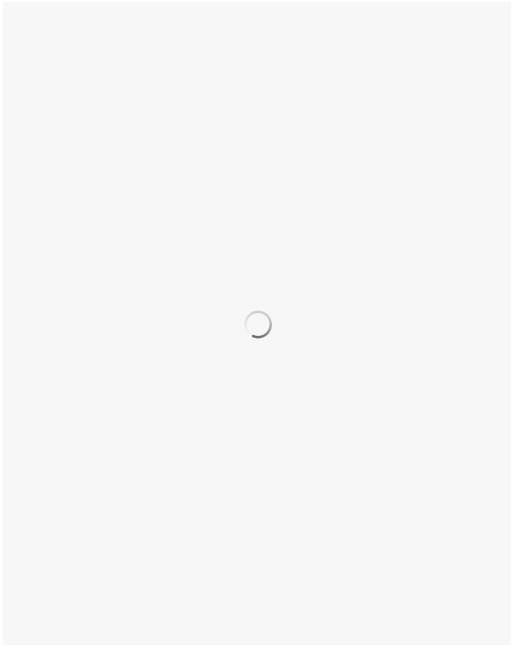
上面说的都是一些准则类的经验，下面将结合具体场景，看一下如何把这些规则落地。以及帮助大家了解一下，除了纯粹的写代码，AI 还可以帮我们做什么。

3.1 快速熟悉项目 & 自然语言检索代码

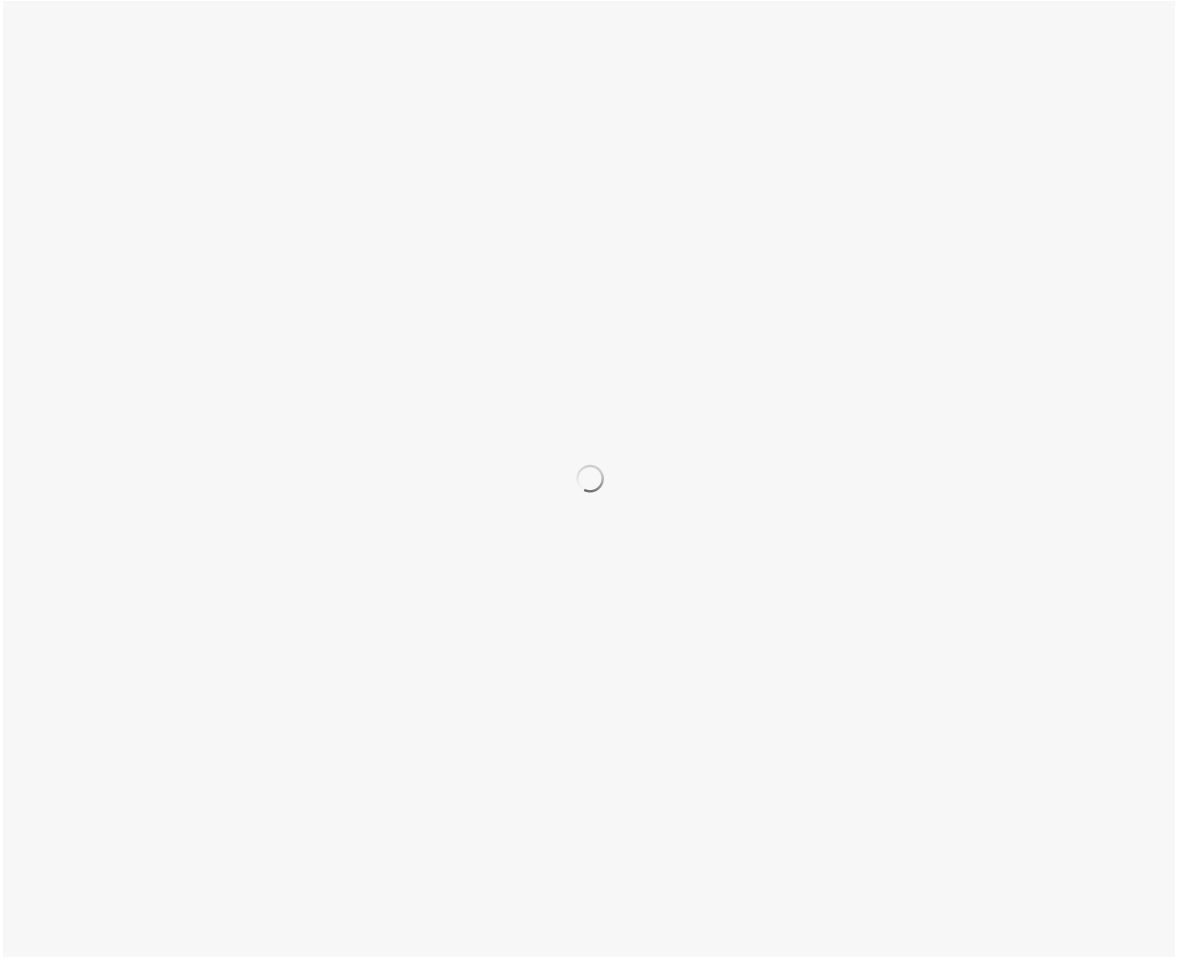
```
讲解一下这个项目的每个module都是用来做什么的，并且给出包依赖关系图。
我希望实现一个「什么什么」功能，需要修改的部分包括「这里」和「那里」，我的代码放在哪个
我在找「某个某个」功能的实现，请帮我在仓库里搜寻，并给出它的核心具体代码位置和片段，并
```

3.2 PlantUML / Mermaid 文本绘图生成

这里是打算让AI帮忙画一下某个功能的前后端交互流程图，先是把接口请求顺序在浏览器里进行了截图：

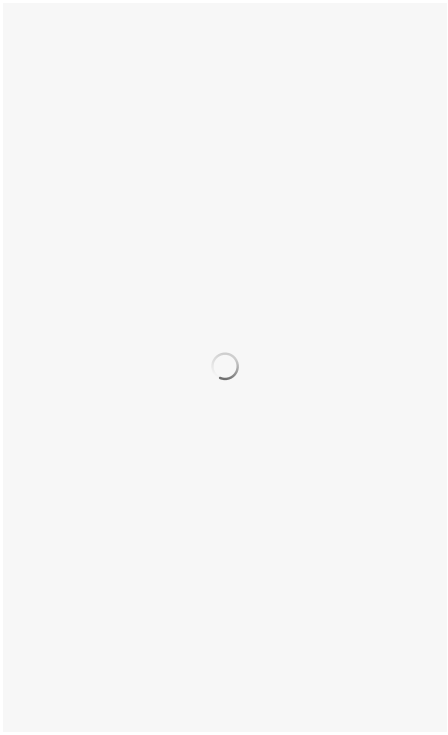


然后把我的需求+我的判断（比如图里有一些接口实际上和这个需求并不相关）一起告诉AI：



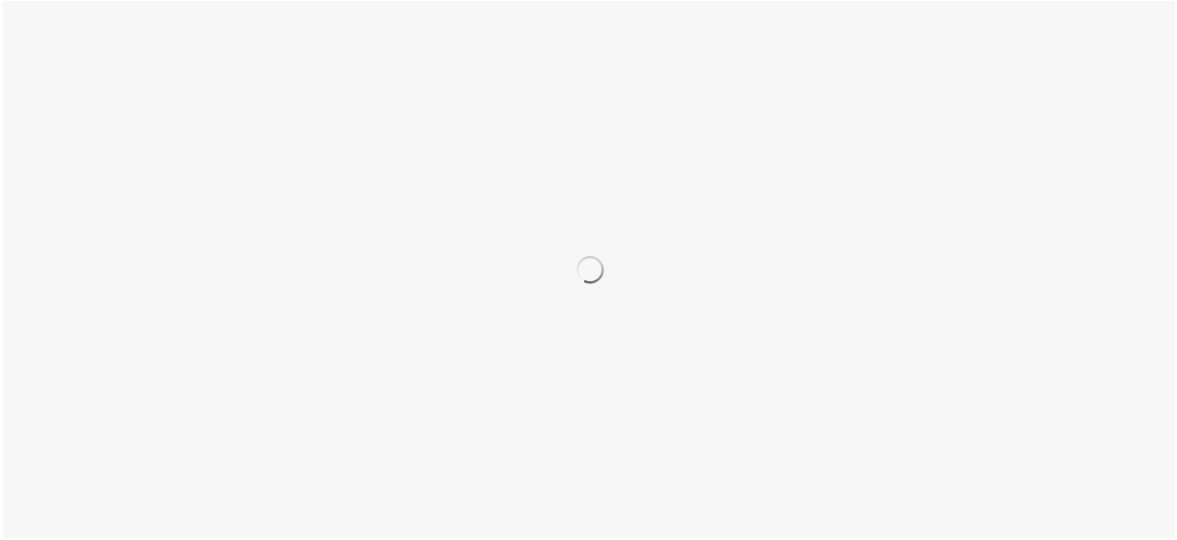
AI在进行了一长串分析之后，也是给出了比较准确的流程图。

结果展示（做了模糊处理）：

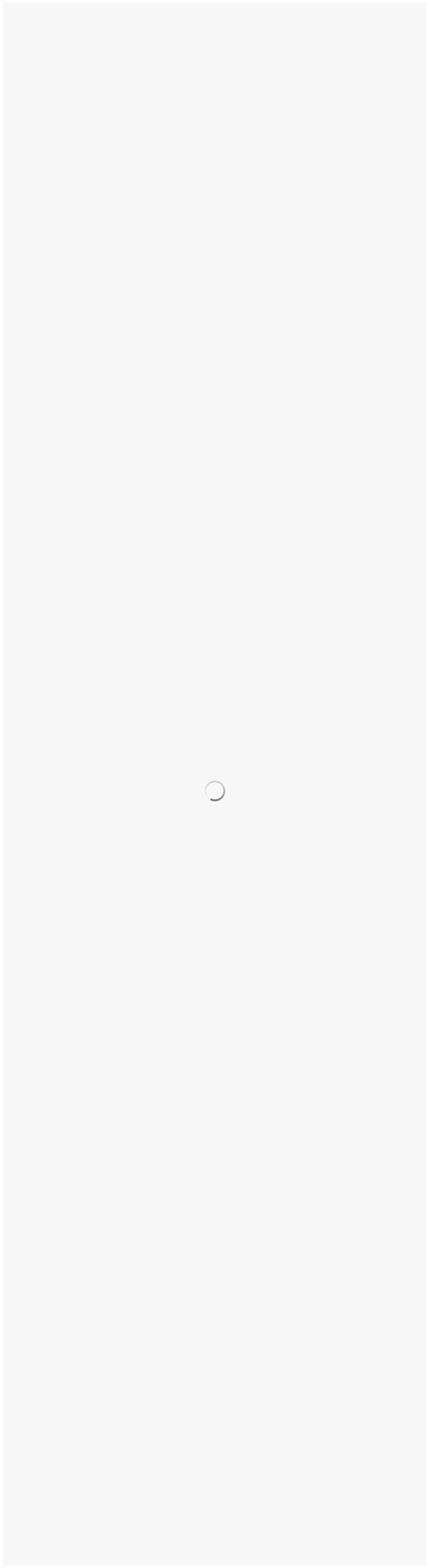


3.3 问题排查

这里是用真实遇到的一个问题举例子，这张图片是执行流程图的展示，当时遇到的问题是这个流程的产出结果出现了问题，大致定位到是图里的左侧节点有问题：



由于对这个仓库并不熟悉，于是暂时交给AI帮忙翻代码排查：

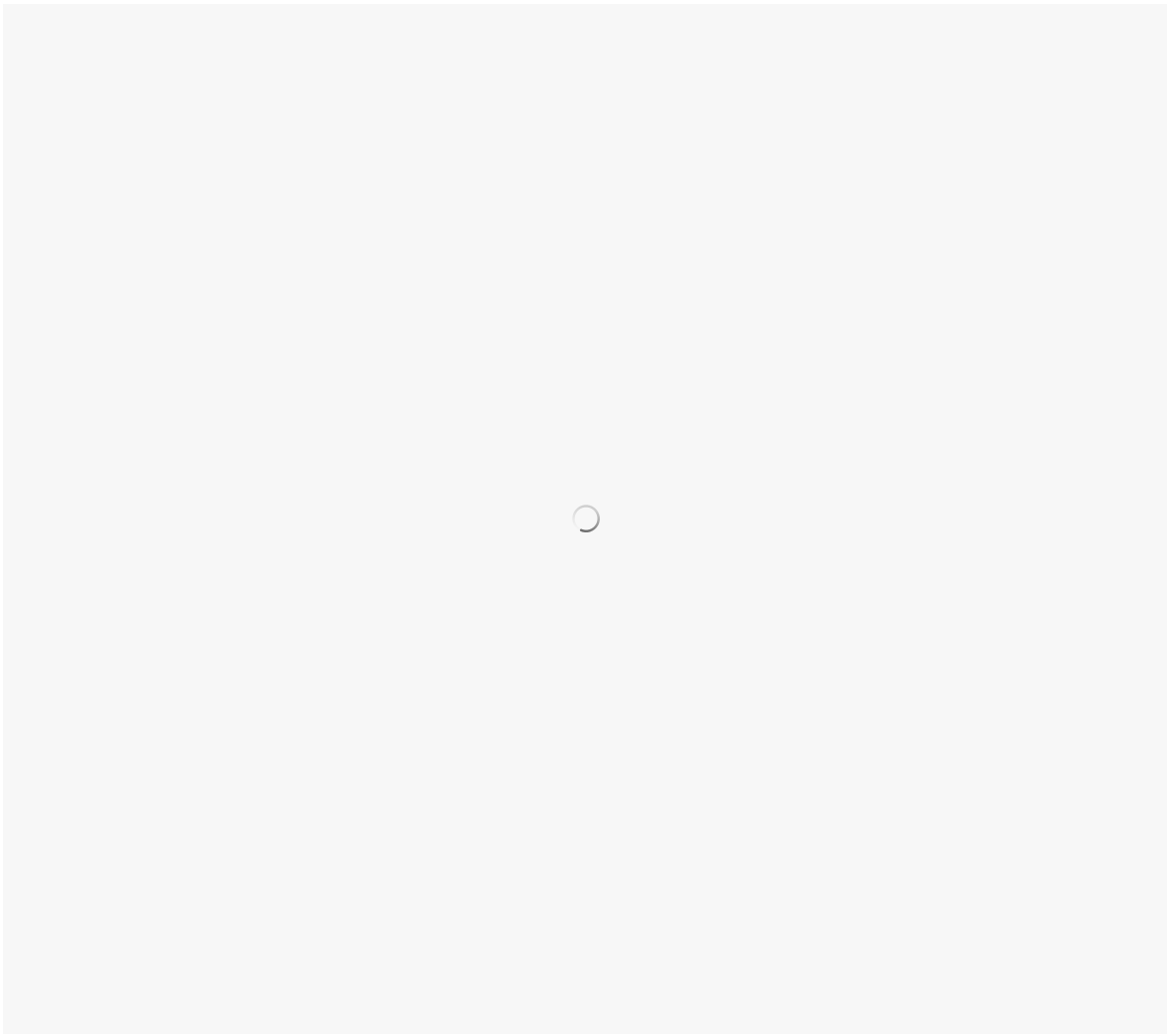


最后人工验证了一下，给出的问题分析和解决方案确实非常详细且准确。在对于项目不熟悉的情况下，这是一种快速进行检索和排查的有效手段。

3.4 补充网页信息到上下文

Web 是相对来说比较容易忽视，但是又非常好用的工具，可以通过@Web的进行添加，也可以直接粘链接进去。需要注意的是，需要进行权限认证的网页（如内网）无法直接被读取。

这里直接粘贴链接进去，让大模型帮忙总结文章内容，也可以基于它直接对任意网页进行问答。

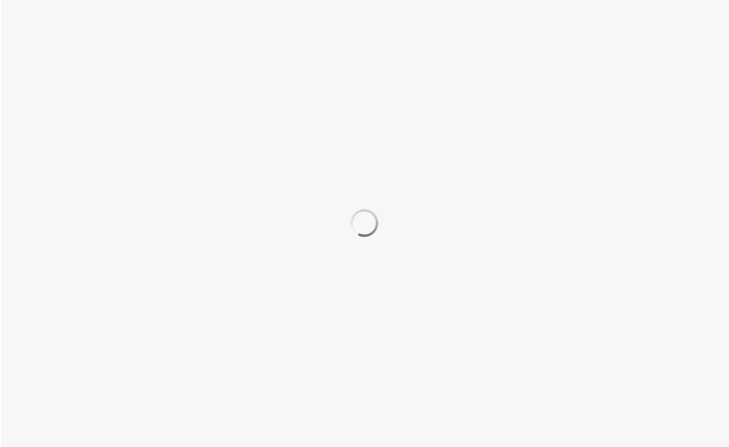


推荐的 Coding 方式

这里主要是一些实践性的探索，目前尚未进行推广，原因是还没有找到一种提升大、好维护、可移植的工具，团队同学的偏好也各不相同。当然，在这方面，团队内部已经有很多探索，目前在积极进行投入和尝试。

4.1 rule 的制定和优化

最基本应该包含你当前项目的技术栈使用，以及对应依赖版本；除此之外应该包含编码明确要求的规范。



注：使用一套统一的rule，需要统一使用cursor，可能不符合个人习惯。但是在使用其它 AI 编程工具时，维护一个项目规则文档，并在对话时手动添加至上下文，可以达到一样的效果。

4.2 文档

根目录下，存放以下内容

- README.md
  - 仓库门面，包含：简介、核心特性、快速开始、其它重要文件的说明和索引
- CHANGELOG.md
  - 需要发版的仓库可以考虑添加
  - 包含：
    - 新增功能
    - bug修复
    - breaking changes（破坏性变更）
    - 注意事项等
- ARCHITECTURE.md
  - 架构复杂且需要多人协作的仓库可以考虑添加
  - 包含：
    - 整体架构
    - 模块划分
    - 核心逻辑流程图

在项目根目录中建立/docs目录

- 目的：让团队成员快速理解项目、高效协作
- 内容：兼顾实用性、准确性、易维护性
- 可以包含以下内容文件
  - 错误码体系
    - 错误码格式
    - 错误码分类
    - 详细场景
  - 异常处理指南
    - 可重试/不可重试
    - 异常抛出原则
    - 日志打印要求
    - 常见异常案例
  - 常用工具类

注：需根据项目内容修改，以上仅为个人推荐

4.3 注释和命名

1. 方法、参数等注释，尽量保证语义清晰、内容完整；
2. 可以考虑添加调用方的使用场景；
3. 适当添加行内注释，明确每个分支的场景和期望处理方式；
4. 命名需要清晰，无歧义；
5. AI 生成量 > 80%的文件，建议使用@author AI Assistant，注明是AI生成，便于维护和统计。

4.4 安全合规问题

需要根据公司规定，合规使用。

4.5 推广方式

这里是在团队内部创建了一个代码仓库，作一下示例，主要维护这些内容（仍在优化中）：

- 范围确定
  - 核心代码库梳理
  - 区分给人看的文档（详细）和给LLM看（核心内容）的文档

- Docs资产
  - 内容指定：markdown文档内容by project定制
  - 生产：初始化文档，暂定为各应用owner
- Project Rule
  - 内容指定：rule内容by project定制，产出为markdown；放在仓库中
  - 生产：初始化project rule的md文档，暂定为各应用owner
- User Rule
  - 内容指定：产出通用部分，如团队规约，保证内容简洁；不放在仓库中
  - 同步方式：（暂定）统一维护在一个新增仓库中，各应用中可以使用脚本进行一键同步到本地

参考文献

- 《Cursor 实战万字经验分享，与 AI 编码的深度思考》：https://www.cnblogs.com/echolun/p/18965624
- 《Claude Code（及 cursor） 内部工作原理窥探》：https://www.superlinear.academy/c/share-your-projects/claude-code-cursor

团队介绍

本文作者库啵，来自淘天集团-用户&内容技术团队。本团队是双边电商平台的核心驱动力之一，通过精细化运营、数据智能和技术创新，实现用户增长与内容价值最大化。同时我们也在持续跟进前沿技术，探索如何让技术在真实业务场景中落地，形成可复用的增长方法论与工程实践。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术
大淘宝技术官方账号
967篇原创内容
公众号

阅读 1.5万